

AI가 모르는 C# 프레임워크로 Mac에서 메모장 만들기

.NET Dev

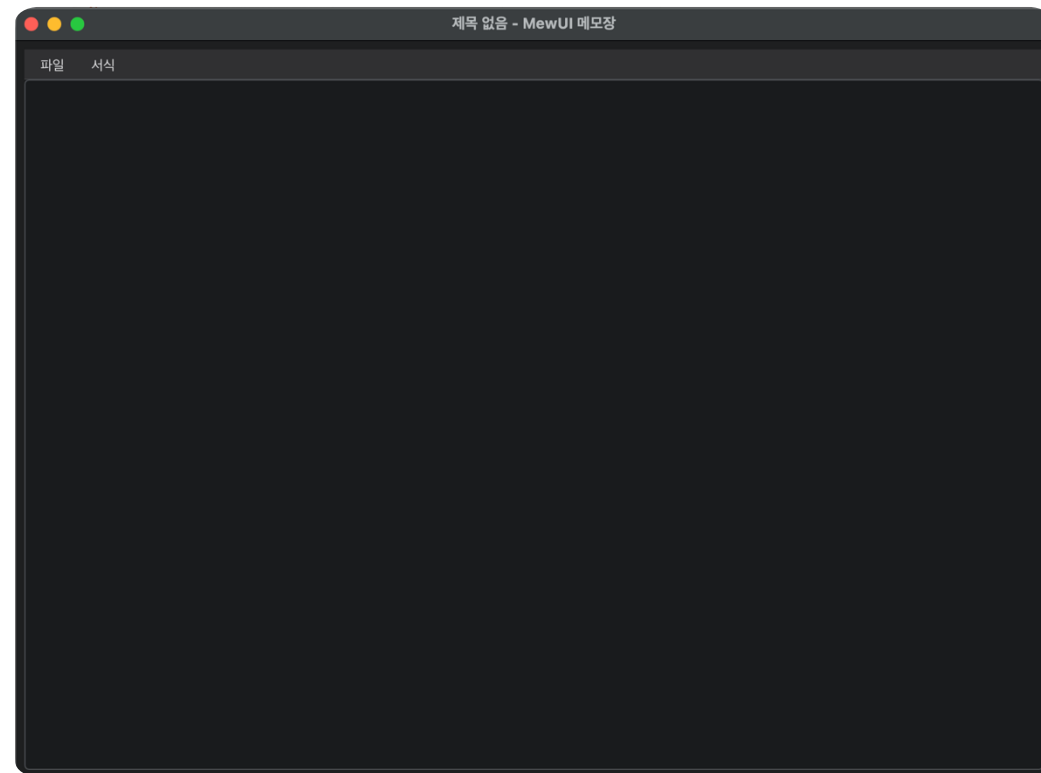
남정현

Swift를 몰라도 Mac용 앱을 만들 수
있습니다.

바이브코딩의 시대

이번에 만든 것

닷넷데브 운영진으로 같이 활동하고
계신 송영재님의 크로스플랫폼 GUI
프레임워크인 MewUI를 바이브코딩
방식으로 활용해서 만든 메모장 앱니다.



만들기 위해서 사용한 것

코드를 처음 스캐폴딩하는 단계에서
어떤 IDE나 코드 에디터로 사용하지
않았습니다.

항목	내용
언어	C# (AI의 재량으로 Swift 일부)
프레임워크	Aprillz.MewUI 0.19.1
편집기	없음
프로젝트 파일	Notepad.csproj
사람이 개입하여 고친 코드	0줄

MacOS 이해하기

일단 MacOS의 애플리케이션 구조를 알아야 이해가 쉽습니다.

먼저 짚고 넘어갈 것

- **macOS:** 상호작용 대상은 **App**이고, App이 사용자와 만나기 위해 **Window**를 띄우며, Window 내부의 모습은 **View**가 구현합니다.
 - 그래서 macOS에서는 창을 모두 닫아도 프로그램이 살아있을 수 있고, 창이 없어도 상단 메뉴 막대는 그 앱의 것으로 남습니다.

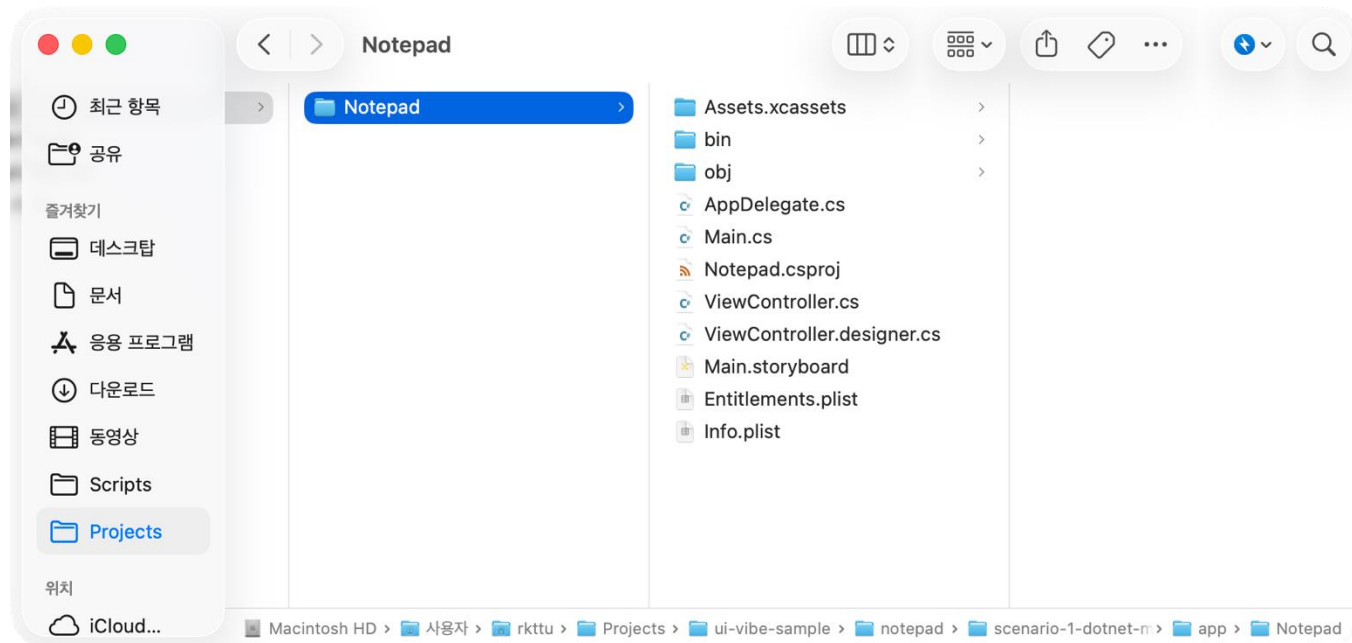
먼저 짚고 넘어갈 것

- **Win32:** 상호작용 대상은 **Window**이고, 그 Window가 내부를 채우기 위해 **또 다른 Window**를 자식으로 만들며, 그 자식들 역시 **똑같은 Window**입니다.
 - 그래서 Windows에서는 마지막 창을 닫을 때 종료하도록 **개발자가 직접 배선했습니다.** 살아 있게 유지할 앱 객체가 없기 때문입니다.
 - 창 안에 창을 띄우는 MDI는, 없는 앱 층을 앱 안에서 흉내 낸 시도였습니다. Alt+Tab은 앱 하나로 묶고 Ctrl+F6으로 내부 창을 넘기는 구조가 ㄹTab / ㄹ 와 같습니다. 현재는 거의 쓰이지 않습니다.

.NET for MacOS

프로젝트 구성

- AppDelegate.cs ➔ 앱 시작, 종료, 파일 열기 요청 등을 받는 곳
- ViewController.cs ➔ 앱 화면 관리자
- Main.storyboard ➔ 창, 메뉴, 뷰 배치를 정의하는 파일
- Info.plist, Entitlements.plist ➔ 애플리케이션 속성, 권한 정의
- Main.cs ➔ NextStep (NS) API를 부르기 위한 애플리케이션 시작점 (우리가 아는 Main 메서드와 동일)



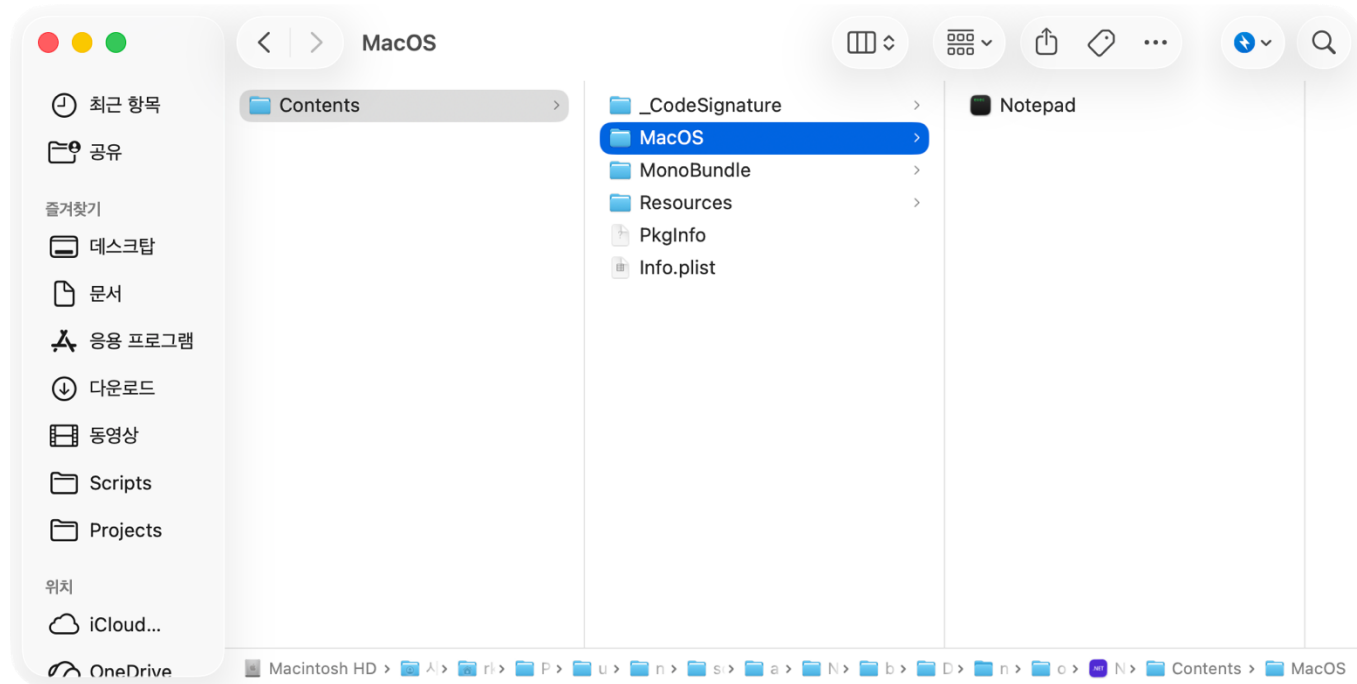
.NET for MacOS 프로젝트 구성

- TargetFramework (TFM)에 -macos라는 접미사를 붙입니다.
- SupportedOSPlatformVersion에는 최소 지원할 macOS 버전을 기재합니다.
- 나중에 Apple App Store로 내보내는 것까지 고려한다면 AOT 빌드도 필요합니다.

```
notepad > scenario-1-dotnet-macos-workload > app > Notepad > Notepad.csproj
1  <Project Sdk="Microsoft.NET.Sdk">
2    <PropertyGroup>
3      <TargetFramework>net10.0-macos</TargetFramework>
4      <OutputType>Exe</OutputType>
5      <Nullable>enable</Nullable>
6      <ImplicitUsings>true</ImplicitUsings>
7      <SupportedOSPlatformVersion>12.0</SupportedOSPlatformVersion>
8
9    <!--
10     | Enable full trimming in Release mode.
11     | To learn more, see: https://learn.microsoft.com/dotnet/core/deploying/trimming/tri
12     -->
13    <TrimMode Condition="'$(Configuration)' == 'Release'">full</TrimMode>
14  </PropertyGroup>
15 </Project>
16
```

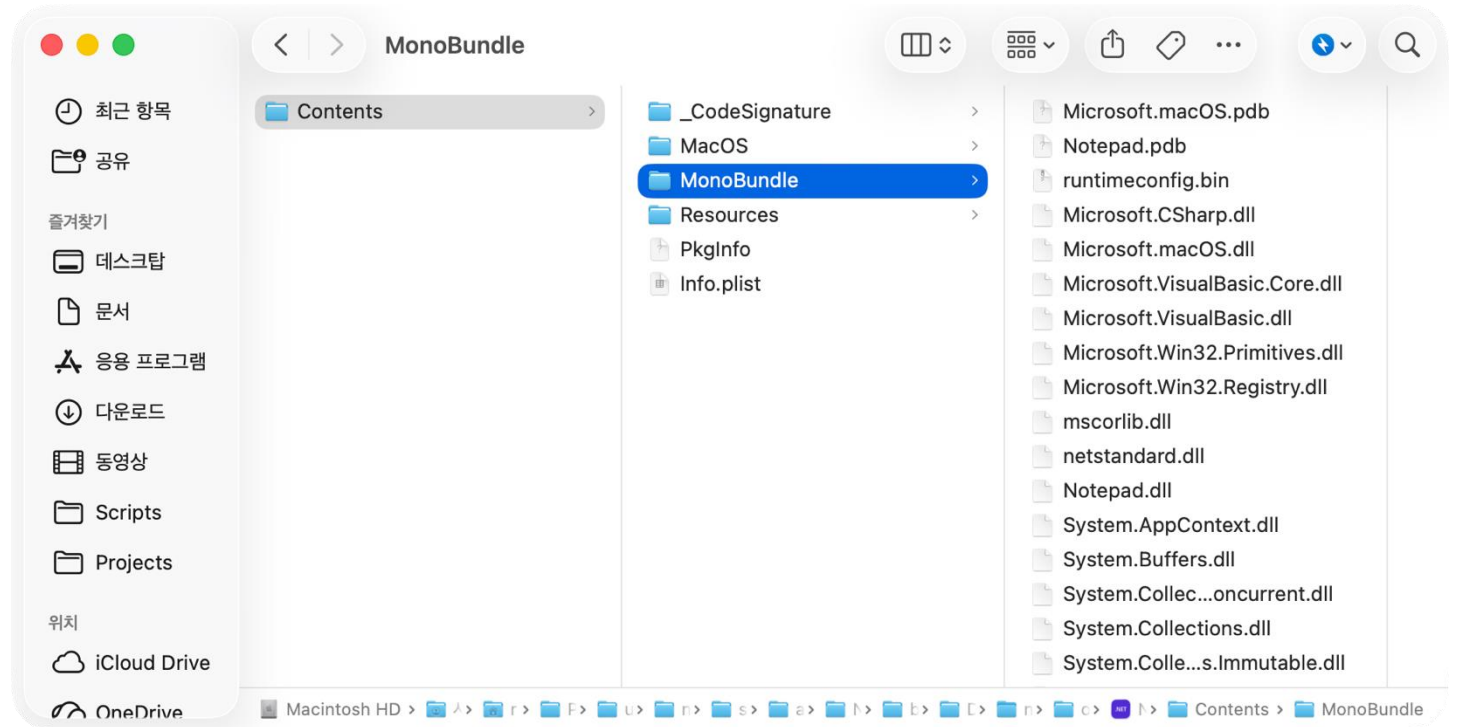
.NET for MacOS 빌드 구성

- 콘솔 프로그램과 달리 MacOS용 GUI 애플리케이션은 디렉터리 이름 뒤에 .app 이라는 접미사를 붙이고 이 안에 애플리케이션 실행에 필요한 파일을 모두 넣습니다.
- Contents/MacOS 디렉터리 안에 실제 네이티브 실행 파일이 들어있습니다.
- DMG, PKG 파일로 MacOS용 앱을 패키징하는 이유가 이 때문입니다.
- DMG의 경우 앱 아이콘을 '응용 프로그램' 폴더에 던져 넣는 방식이 제공되는 이유 역시 이와 관련이 있습니다.
- 이것만으로 충분치 않은 부가 설치 동작이 있는 경우 설치 관리자를 추가로 사용하게 됩니다.



.NET for MacOS 빌드 구성

Contents/MonoBundle 디렉터리
안에 .NET 관련 런타임이 들어가며,
나중에 AOT 빌드를 거치면서 모두
네이티브 실행 파일로 병합됩니다.



Demonstration

만들어진 코드 살펴보기

HandMirrorMcp 서버 이야기

이번에 알게 된 것

.NET for macOS 바이브코딩 프롬프트 (s1)

.NET macOS 워크로드(AppKit)를 사용해서 macOS 네이티브 메모장 앱을 만들어주세요.

모든 산출물은 반드시 ./app/ 디렉터리 안에만 만들고, 그 바깥에는 어떤 파일도 만들거나 수정하지 마세요.

여러 줄 텍스트 편집 영역이 창을 채우고, 파일 열기와 저장이 되어야 합니다. 자동 줄바꿈을 켜고 끌 수 있고, 저장하지 않은 변경이 있는 상태에서 창을 닫으려 하면 확인을 받아야 합니다. 없는 파일을 열거나 권한이 없는 곳에 저장하는 등 오류 상황에서도 앱이 죽지 않고 오류를 표시해야 합니다.

웹 검색과 웹 페치는 차단되어 있으니, 모르는 API나 프로젝트 설정은 Microsoft Learn MCP 서버로 확인하면서 진행해 주세요. 추측으로 API 이름을 지어내지 말고, 근거가 없으면 먼저 문서를 조회해 주세요.

마지막에 dotnet build 가 통과하고 .app 번들이 생성되는 것까지 확인해주세요.

MAUI 바이트코딩 프롬프트 (s2, s3)

Aprillz.MewUI 프레임워크로 크로스플랫폼 데스크톱 메모장 앱을 만들어주세요.

모든 산출물은 반드시 ./app/ 디렉터리 안에만 만들고, 그 바깥에는 어떤 파일도 만들거나 수정하지 마세요.

여러 줄 텍스트 편집 영역이 창을 채우고, 파일 열기와 저장이 되어야 합니다. 자동 줄바꿈을 켜고 끌 수 있고, 저장하지 않은 변경이 있는 상태에서 창을 닫으려 하면 확인을 받아야 합니다. 없는 파일을 열거나 권한이 없는 곳에 저장하는 등 오류 상황에서도 앱이 죽지 않고 오류를 표시해야 합니다.

웹 검색과 웹 페치는 차단되어 있으니, 사용할 수 있는 MCP 서버만으로 API를 확인하면서 진행합니다. 추측으로 API 이름을 지어내지 말고, 근거가 없으면 먼저 조회합니다.

macOS에서 dotnet run 으로 빌드가 통과하고 창이 실제로 뜨는 것까지 확인해주세요.

MCP 서버 구성 비교

S2

```
"mcpServers": {  
  "microsoft-learn": {  
    "type": "http",  
    "url":  
    "https://learn.microsoft.com/api/mcp"  
  }  
}
```

S3

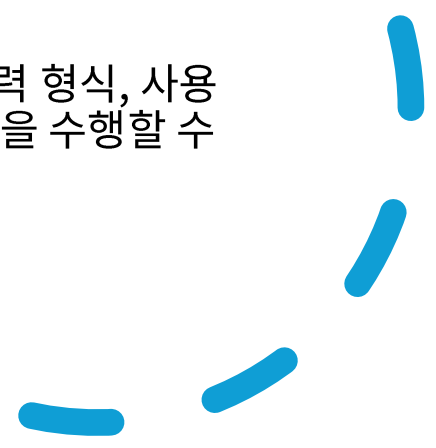
```
"mcpServers": {  
  "microsoft-learn": {  
    "type": "http",  
    "url":  
    "https://learn.microsoft.com/api/mcp"  
  },  
  "handmirror": {  
    "type": "stdio",  
    "command": "dotnet",  
    "args": ["dnx", "HandMirrorMcp@0.1.1"]  
  }  
}
```


bash

```
for repo in $(cat targets.txt); do
  claude -p "$(envsubst < task.tmpl)" \
    --bare --model claude-sonnet-5 --strict-mcp-config \
    --mcp-config ./tools.json --max-turns 30 --max-budget-usd 1.00 \
    --output-format json > "out/${repo}.json"
done
```

여러 번 바이브코딩 실행하기

- Claude Code CLI에서 잘 조명되지 않는 기능 중 파라미터 지정 기능이 있습니다.
- 우측의 그림과 같이 원하는 MCP 서버 설정과 회차, 출력 형식, 사용 가능한 예산 한도 지정 기능을 통해 반복적인 바이브 코딩을 수행할 수 있습니다.



바이트코딩 실험 결과

- Sonnet 5, Opus 5로 .NET for MacOS (NSAPI), MewUI 기반 MacOS 데스크톱 앱을 만드는 것은 잘 작동했습니다.
- 반면 Haiku 4.5는 작업을 자체적으로 완결 짓지 못하고 권한을 요청하거나 사람에게 미완성 코드를 넘기는 경향이 컸습니다.

바이트코딩 실험 결과

시나리오	모델	결과	시간	출력 토큰	도구 / MCP 호출 횟수	특징
#1	Opus 5	성공	18.2분	192,125	57 / 13	스크린샷 1, 프로세스 확인 20
#1	Sonnet 5	성공	15.8분	184,059	125 / 39	시간·토큰 모두 Opus 아래(차이는 15% 문턱 미만). 대신 도구 2.2배·MCP 3.0배
#1	Haiku 4.5	무효	2.5분	34,132	32 / 9	권한 거부 5건, 셀 차단 5건
#2	Opus 5	성공	17.4분	143,524	98 / 2	여기서도 MCP 2회로 끊고 산출물 근거로 전환. 스크린샷 6, 이미지 확인 6
#2	Sonnet 5	성공	18.4분	198,842	94 / 6	토큰은 Opus의 1.4배. 스크린샷 6, 프로세스 확인 7
#2	Haiku 4.5	실패	0.6분	5,783	7 / 6	33초. 프로젝트조차 만들지 않고 "지금 진행하시겠습니까"로 종료. 권한 거부·셀 차단 0건이라 능력만 측정된 유일한 Haiku 셀
#3	Opus 5	성공	18.8분	213,200	157 / 72	전 회차 최대 검증 밀도 — 스크린샷 18, 이미지 확인 17, 프로세스 확인 25
#3	Sonnet 5	성공	16.3분	209,654	137 / 82	전 회차 최다 조회. 스크린샷 10, 이미지 9, 프로세스 19
#3-2	Haiku 4.5	실패	6.0분	104,872	70 / 45	안내 한 문단으로 not found 29건 → 0건, 정답 타입 모두 발견. 그럼에도 코드를 사람에게 넘기고 요구사항 절반을 미룸

바이트코딩 실험 결과

- Opus 5, Sonnet 5는 모두 바이트코딩에 성공했지만, Haiku 4.5는 성공하지 못했습니다.
- MCP 호출 횟수는 Sonnet이 Opus보다 많았습니다. "몇 번 물어보고 확신하는가"의 차이를 그대로 보여줍니다.
- **Opus**는 근거를 일찍 확정된 후 남은 시간을 검증에 쓰고, **Sonnet**은 확정을 늦추며 그만큼을 조화에 쓰는 경향을 보입니다.

HandMirrorMcp 사용 여부에 따른 차이

쌍	프로브 작성 (s3)	검증량 변화 (s2 → s3)	토큰 사용량 증가
계산기 Opus	생략	감소 (엔진 22종 + 클릭 6종 → 클릭 4종)	+5.3%
계산기 Sonnet	생략	증가 (검증 중 버그 발견 → 수정 루프)	+27.7%
메모장 Opus	생략	크게 증가 (스크린샷 6 → 18)	+48.5%
메모장 Sonnet	생략	증가 (스크린샷 6 → 10, 프로세스 7 → 19)	+5.4%

HandMirrorMcp에 대한 교훈

- HandMirrorMcp 도구를 쓴 회차는 탐색 코드 작성을 건너뛰었습니다.
- 그런데 그렇게 아낀 시간과 비용이 HandMirrorMcp와의 빈번한 도구 호출로 인해 상쇄되는 것으로 보였습니다.
- 무엇이 원인인지 따져보니, 결국 HandMirrorMcp의 모든 일들은 Shell 작업으로 대체가 가능한 것들이었습니다.
- Shell을 활용하여 작업할 수 있는 것들은 코딩 에이전트에게 맡기는 것이 토큰 효율이 더 좋았습니다.

덮이는 것, 덮으면 안 되는 것, 닿지 않는 곳

셀이 덮는다

CLI와 스킬로 간다

여기에 서버를 만들면 상시 컨텍스트 비용만 남는다.

로컬 파일·패키지·프로세스 조사
한 번 만들게 하고 CLI로 굳힌다

REST API 조회와 조작

`curl -H "Authorization: Bearer ..."` 한 줄

쿠키 세션 유지

`curl -c jar.txt -b jar.txt` — 상태는 프로세스가 아니라 파일에 둔다

OAuth의 기계적인 부분

device code, client credentials, refresh 전부 그냥 POST

브라우저 탐색

헤드리스 + `--user-data-dir` 로 프로필 고정

절차와 노하우

스킬 파일. 릴리스 없이 고쳐진다

일회성 탐색 도구

모델이 직접 쓴다. 쓸 만하면 그때 굳힌다

덮으면 안 된다

경계를 파는 자리

셀로 할 수 있지만, 할 수 있게 두면 목적이 무너진다.

자격 증명

모델이 CLI를 부를 수 있으면 그 CLI가 읽는 환경 변수도 읽는다. 여기서 `curl` 은 대안이 아니라 반대편이다

우회 가능하면 무의미한 정책

읽기 전용 강제, 행 수 상한, 마스킹 — 대체 경로를 달는 것까지가 설계다

완전성이 요구되는 감사 기록

한 경로라도 새면 기록이 아니라 표본이 된다

사람의 동의를 필요한 순간

셀은 URL을 열 수는 있어도 승인 버튼을 누르지 못한다

셀이 없다

닿지 않는 표면

가장 강력한 이유이지만, 가장 흔하지는 않다.

웹·모바일 클라이언트

Bash가 없다. 우회 논의 자체가 성립하지 않는다

커넥터·채팅 표면

호스트가 인증과 세션을 대신 관리한다

도구 설치가 통제된 환경

사용자가 설치하지 못하고 관리자가 엔드포인트로 제공한다

살아 있어야 하는 연결

구독 중인 스트림, 붙어 있는 디버거. 다만 로컬 데몬 + CLI로도 된다

무엇을 어디에 담는가

담을 것	그릇	이유
로직	CLI	모델도 부르고 사람도 부르고 CI에도 들어간다. 상시 컨텍스트 비용이 없다
노하우	스킬 파일	산문은 산문의 수명을 갖는다. 한 문장 고치려고 릴리스를 내지 않는다
경계	MCP 서버	자격 증명 격리, 정책 강제, 감사. 우회로가 닫혀 있을 때만 성립한다
검증	없음	어느 그릇도 채워주지 않는다. 실행 증거를 요구하는 수밖에 없다

흔한 오판

REST API를 감싼 서버

셀이 있는 환경에서 이건 `curl` 한 줄이다. 엔드포인트 목록과 호출 순서는 스킴에 적으면 되고, 그쪽이 컨텍스트 비용도 0이다.

모델의 부족함을 메우려는 조회 서버

정보가 산출물에서 기계적으로 나오고 대체 경로가 저렴하면 수명이 짧다. 모델이 그 일을 하게 되거나, 안내 몇 줄이 자리를 대신한다.

노하우를 도구 설명에 박아두기

“정확한 이름 대신 와일드카드를 먼저 써라”를 고치려고 릴리스를 내게 된다. 같은 내용이 마크다운에서는 한 줄 수정이다.

셀을 열어둔 채 서버로 정책 강제

모델이 같은 자원에 다른 경로로 닿을 수 있으면 격리도 정책도 성립하지 않는다. 계약이 아니라 권고가 된다.

HandMirrorMcp에 대한 교훈

- 그래서 .NET 개발 생태계에 있어 더 좋은 선택지는 현재의 MCP 서버가 아니라는 것을 알 수 있었습니다.
- 대신, 현재의 .NET CLI 도구들이 빈약한 부분을 보완해줄 다른 오픈 소스 CLI를 큐레이션하고 어떻게 쓸지 문서화하는 것이 더 유용하다는 결론에 도달했습니다.
- HandMirrorMcp의 다음 버전을 만든다면 스킴과 도구 중심으로 재편하게 될 것 같습니다.

고맙습니다!

LinkedIn

<https://linkedin.com/in/rkttu>

Code Samples

<https://github.com/rkttu/ui-vibe-sample>

<https://github.com/rkttu/HandMirrorMcp>

<https://github.com/aprillz/MewUI>

.NET Dev Forum

<https://forum.dotnetdev.kr/>